# Quantification de l'Incertitude sur Texte Generatif (ELI5)

Analyse realisee sur le dataset ELI5 (Explain Like I'm 5),

100 questions a reponse ouverte traitees par Gemini 2.0 Flash.

---

## Methodes implementees

1. Perplexite et Entropie top-k (niveau token)
2. LLM-as-Judge (evaluation de qualite)
3. Semantic Entropy (niveau reponse)
4. SelfCheckGPT (verification phrase par phrase)
5. Score de confiance combine + evaluation

---

## Pipeline

Question ELI5 --> Generation (Gemini 2.0 Flash, logprobs)

--> Perplexite + Entropie par phrase

--> K=5 reponses alternatives (temp=1.0)

--> Clustering NLI --> Semantic Entropy

--> SelfCheck phrase-par-phrase

--> Evaluation par juge LLM

--> Score de confiance combine

# Methode 1 : Perplexite et Entropie Top-k

La perplexite mesure la 'surprise' du modele face a sa propre generation. Pour chaque token genere, on recupere la log-probabilite attribuee par le modele :

```
PPL(phrase) = exp( -1/N * sum(log P(t_i | t_<i)) )
```

Une perplexite elevee signifie que le modele hesite sur les mots a utiliser, signe potentiel d'incertitude.

**Entropie top-k** : On calcule aussi l'entropie de Shannon sur les** k=5 tokens les plus probables a chaque position. Une entropie elevee signifie que le modele hesite entre plusieurs alternatives.

---

## Statistiques cles

- 100 questions, 436 phrases au total
- PPL moyenne par phrase : 1.33 (std=0.15)
- Entropie top-k moyenne : 0.523
- Correlation PPL vs accuracy : r=-0.179 (p=0.075)

La perplexite est faiblement anti-correlee avec la qualite : les reponses de mauvaise qualite ont une perplexite legerement plus elevee, mais la relation est faible.

# Perplexite : distributions et lien avec l'accuracy

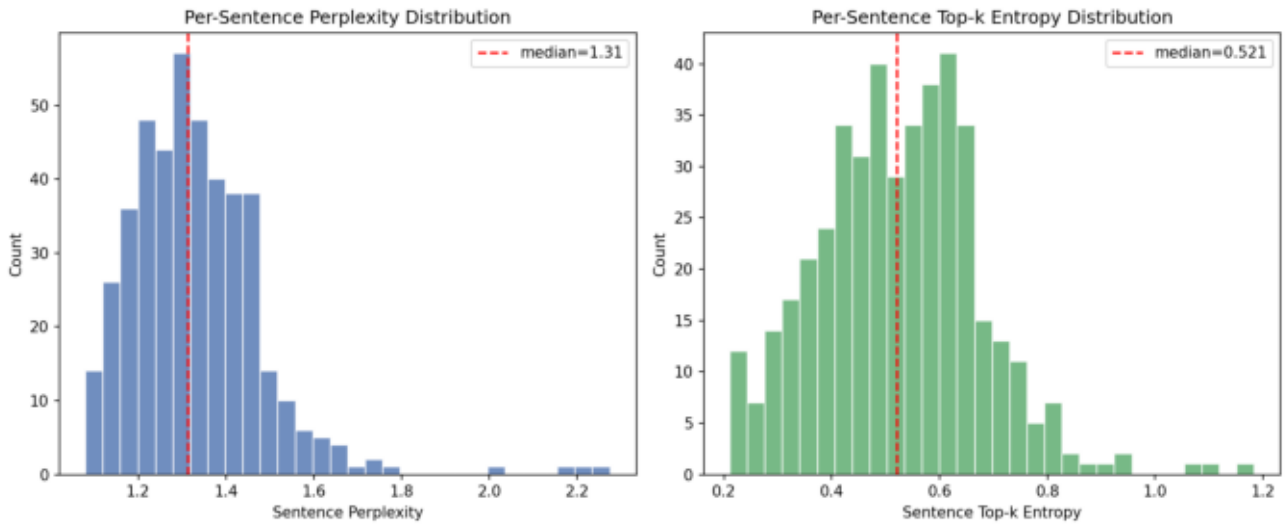


*Figure 1 : Distribution de la perplexite et de l'entropie par phrase*

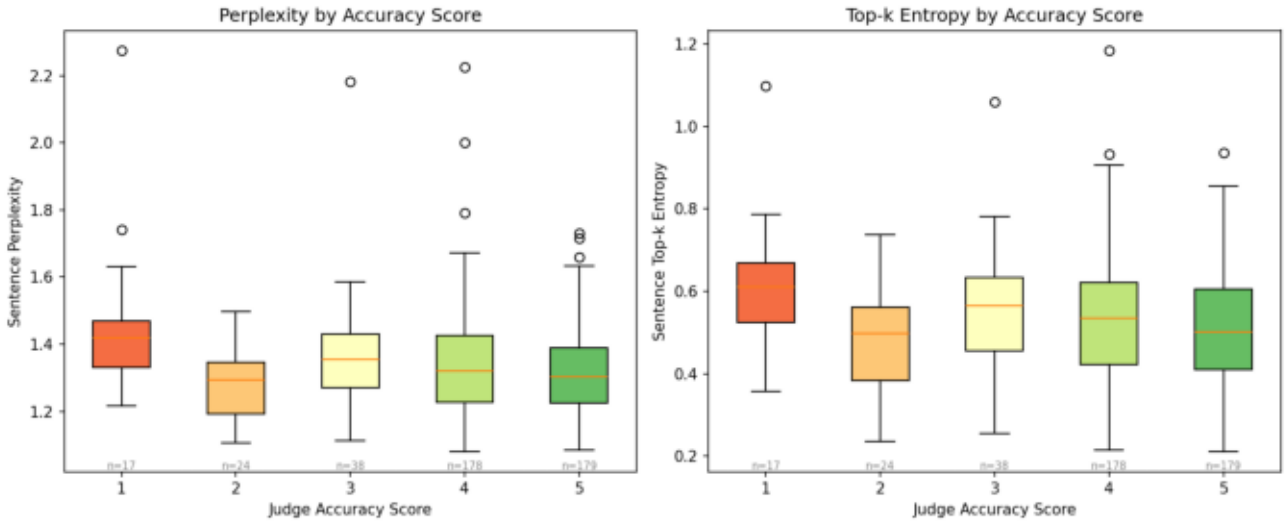


*Figure 3 : Perplexite et entropie groupees par score d'accuracy du juge*

# Exemples de reponses colorees par perplexite

*Figure 4 : Chaque phrase est coloree par sa perplexite (vert=faible, rouge=elevee)*

# Methode 2 : Semantic Entropy

La Semantic Entropy (Kuhn et al., 2023) mesure la diversite semantique des reponses du modele quand on le fait repondre plusieurs fois a la meme question.

## Algorithme

1. Generer K=5 reponses avec temperature=1.0 (haute diversite)
2. Pour chaque paire de reponses, evaluer l'equivalence semantique via NLI (Gemini 2.5 Flash)
3. Regrouper les reponses equivalentes par Union-Find
4. Calculer l'entropie de Shannon sur les clusters :

```
SE = - sum( P(C) * log P(C) )

ou P(C) = sum(exp(mean_log_prob(r))) / Z pour r dans C
```

**SE = 0** signifie que toutes les reponses tombent dans le** meme cluster : le modele est unanime.
**SE > 0** signifie diversite semantique : le modele donne** des reponses differentes selon les tirages.

---

## Resultats

- SE moyenne : 0.3483 (std=0.3992)
- SE = 0 : 51/100 questions (51%)
- Clusters moyens : 1.68
- Correlation SE vs accuracy : r=-0.173 (p=0.085)

Probleme identifie : SE=0 ne garantit pas une bonne reponse.
9 questions sur 51 avec SE=0 ont une accuracy <= 3.

# Semantic Entropy : distribution et lien avec l'accuracy

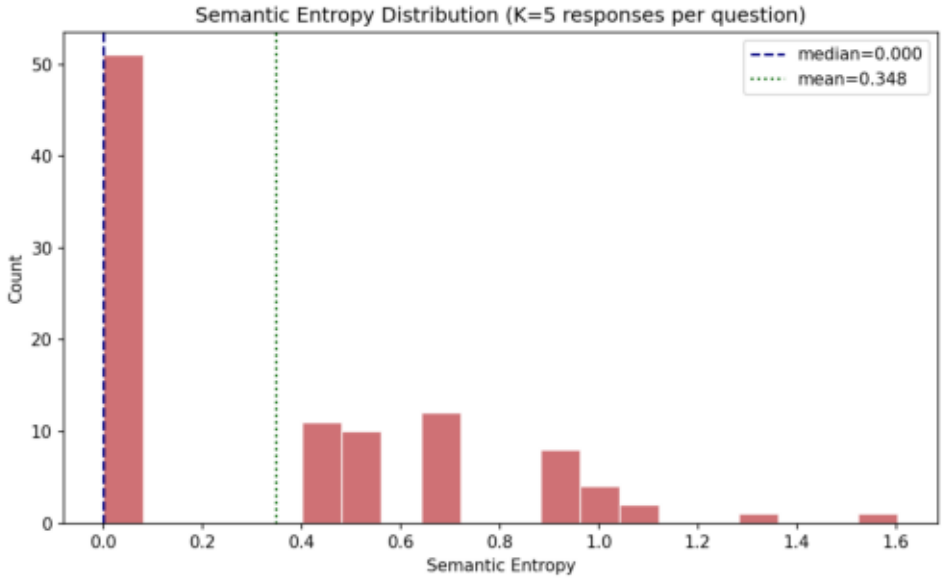


*Figure 6 : Distribution de la Semantic Entropy (51% a zero)*

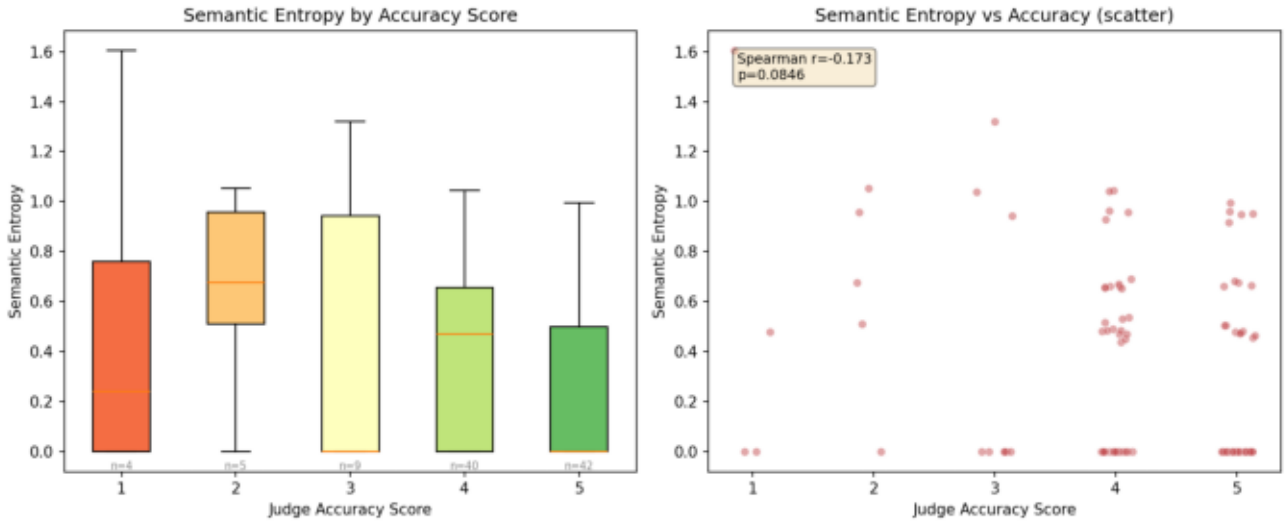


*Figure 7 : SE par score d'accuracy (boxplot + scatter)*

# Semantic Entropy : analyses supplementaires

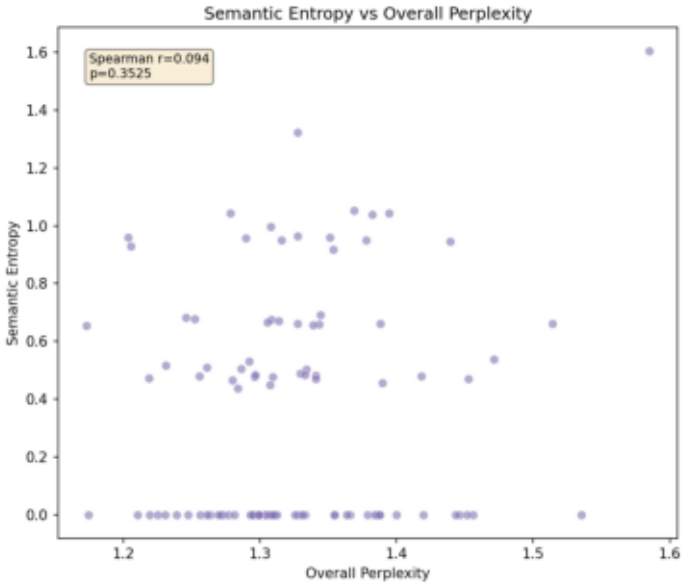


*Figure 8 : SE vs perplexite globale (faible correlation)*

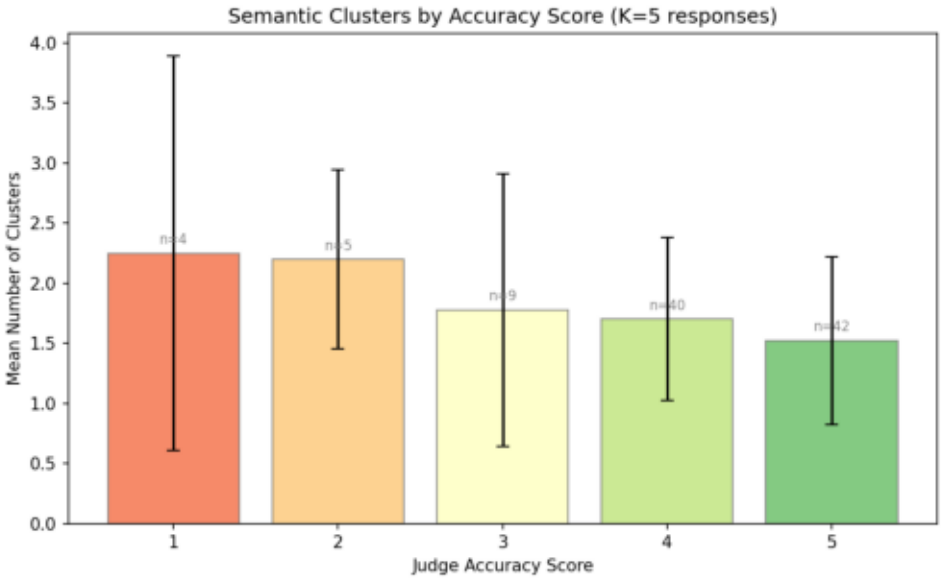


*Figure 9 : Nombre de clusters par score d'accuracy*

# Methode 3 : SelfCheckGPT

SelfCheckGPT (Manakul et al., 2023) verifie la coherence factuelle d'une reponse en la comparant phrase par phrase a des reponses alternatives du meme modele.

## Algorithme

1. Prendre la reponse originale et la decouper en phrases
2. Prendre les K=5 reponses alternatives (de Semantic Entropy)
3. Pour chaque (phrase, passage alternatif), demander a Gemini 2.5 Flash :
   'Cette phrase est-elle supportee par ce passage ?'
4. Score par phrase = proportion de 'Non supporte'
   (0.0 = confirmee partout, 1.0 = contredite partout)
5. Score global = moyenne des scores par phrase

## Adaptation

L'implementation originale utilise un modele HuggingFace local. Notre version utilise Gemini 2.5 Flash via API avec des prompts batchees (toutes les phrases evaluees en un seul appel par passage).

---

## Resultats

- Score moyen : 0.4694 (std=0.1913)
- Fiable (< 0.3) : 16 questions
- Incertain (0.3-0.7) : 67 questions
- Non fiable (>= 0.7) : 17 questions
- Correlation vs accuracy : r=-0.069 (p=0.496)

# SelfCheck : distribution et lien avec l'accuracy

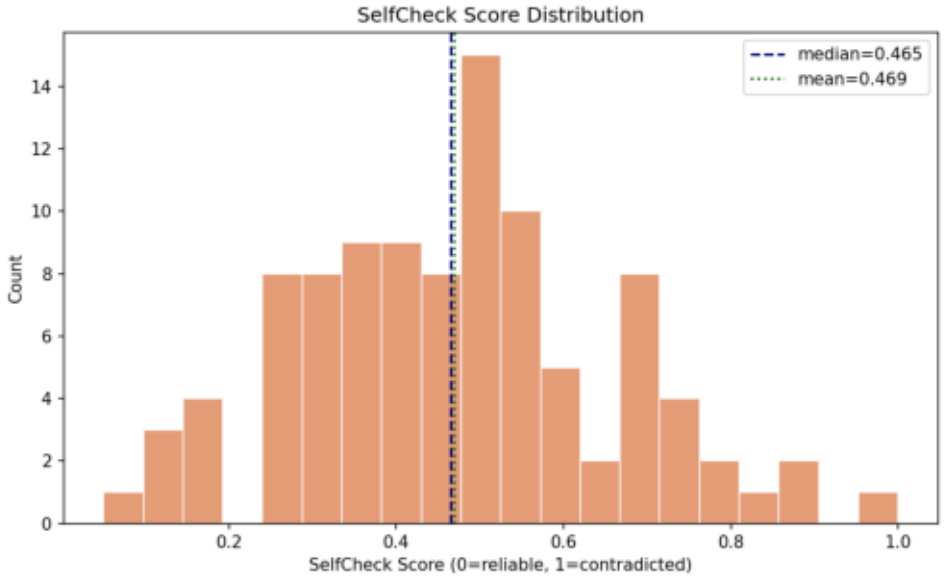


*Figure 10 : Distribution des scores SelfCheck*

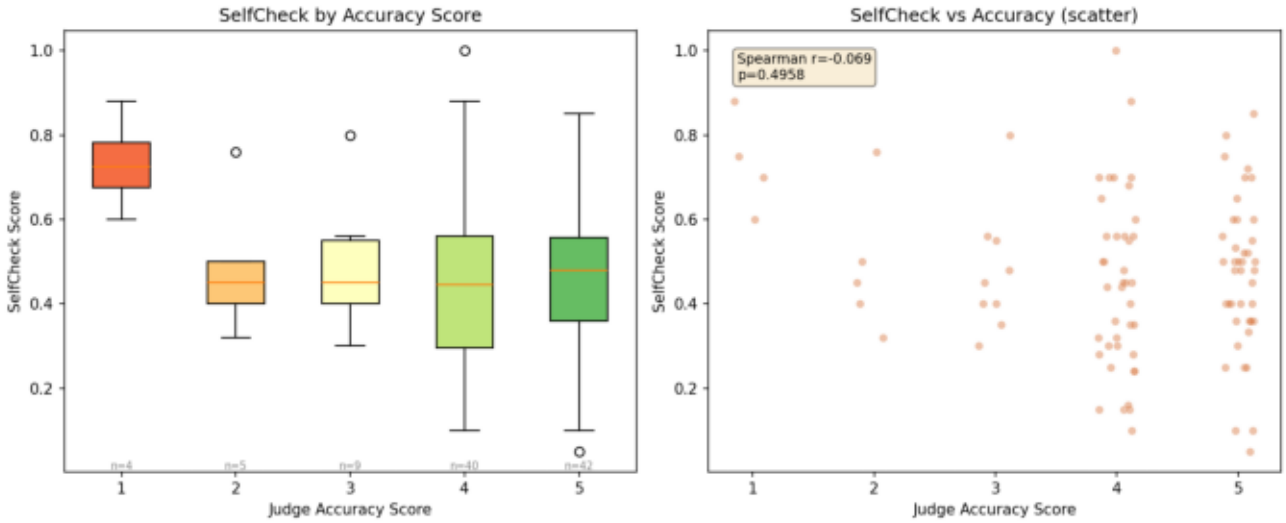


*Figure 11 : SelfCheck par score d'accuracy*

# Score de Confiance Combine

On combine les quatre signaux de confiance pour evaluer
la capacite a predire la qualite des reponses.

## Signaux de confiance

```
c_ppl = 1 - normalized(perplexite)
c_entropy = 1 - normalized(top-k entropy)
c_se = 1 - normalized(semantic entropy)
c_selfcheck = 1 - normalized(selfcheck score)
c_combine = moyenne des 4 signaux
```

Chaque signal est normalise en [0,1] par min-max.
On inverse les signaux ou 'haut = incertain'.

## Evaluation

On definit y=1 si accuracy >= 4, y=0 sinon.
Baseline : 82% des reponses sont de bonne qualite.

## Matrice SE x SelfCheck

On croise SE (confiant si SE=0, incertain si SE>0) avec
SelfCheck (fiable si <0.3, non fiable si >=0.3) pour voir
si la combinaison capture mieux les erreurs.

# Analyse combinee : matrice et courbes de couverture

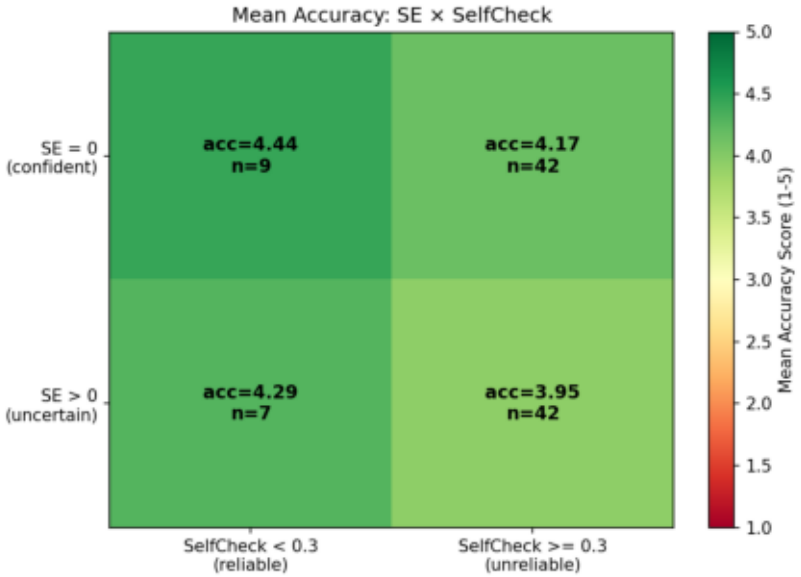


*Figure 12 : Accuracy moyenne par quadrant SE x SelfCheck*

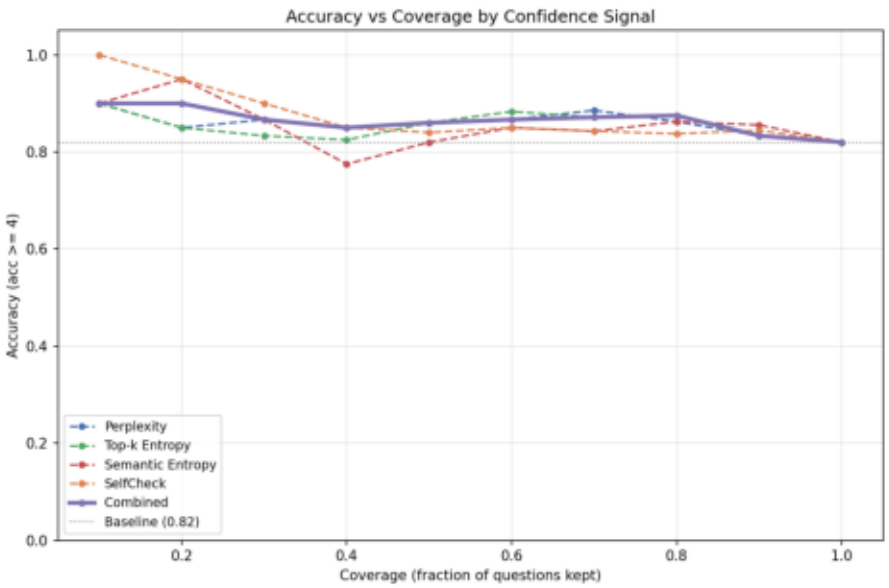


*Figure 13 : Accuracy vs couverture pour chaque signal*

# Tableau des metriques d'evaluation

Baseline : 82% des questions ont accuracy >= 4 (n=100)

*y = 1 si accuracy >= 4, 0 sinon. s = signal de confiance normalise [0,1].*

| Signal | Brier | ECE | AUROC | AUPRC |
|---|---|---|---|---|
| Perplexite | 0.1890 | 0.2015 | 0.6179 | 0.8738 |
| Top-k Entropy | 0.2545 | 0.3066 | 0.6179 | 0.8743 |
| Semantic Entrop | 0.1820 | 0.2206 | 0.5725 | 0.8656 |
| SelfCheck | 0.2312 | 0.2614 | 0.6118 | 0.8888 |
| Combine | 0.1826 | 0.1992 | 0.6247 | 0.8678 |

Interpretation des metriques :

- Brier : erreur quadratique moyenne entre confiance et resultat.
  Plus bas = meilleure calibration.

- ECE : erreur de calibration attendue (binned). Plus bas = mieux.

- AUROC : capacite du signal a distinguer bonnes/mauvaises reponses.
  0.5 = hasard, 1.0 = parfait.

- AUPRC : aire sous la courbe precision-rappel.
  Utile quand les classes sont desequilibrees (82% de positifs ici).

# Conclusions et Perspectives

## Resultats principaux

1. La perplexite et l'entropie top-k capturent une hesitation au niveau des tokens, mais la correlation avec la qualite des reponses reste faible (r=-0.179).

2. La Semantic Entropy mesure la coherence inter-reponses. 51% des questions ont SE=0 (modele unanime), mais cela ne garantit pas la justesse : 9/51 unanimes ont acc<=3.

3. SelfCheck verifie la coherence phrase-par-phrase. La majorite des questions (67%) sont dans la zone 'incertaine' (0.3-0.7), montrant une verification plus nuancee que SE (binaire).

4. Aucun signal individuel ne permet de predire la qualite des reponses avec fiabilite (meilleur AUROC ~ 0.62). Cela est coherent avec la nature du probleme : pour des questions ouvertes, l'incertitude du modele ne correspond pas directement a la justesse factuelle.

5. Le signal combine ameliore legerement la calibration (meilleur Brier et ECE) mais l'AUROC reste limite.

---

## Limites

- Echantillon limite (100 questions ELI5)
- Evaluation par juge LLM (pas de ground truth humain)
- Questions ouvertes : difficile de definir 'correct'
- Le modele peut etre confiant et faux (SE=0, mauvaise acc.)

---

## Perspectives

- Appliquer sur un dataset plus grand / plus diversifie
- Combiner avec conformal prediction pour des garanties statistiques de couverture
- Explorer des methodes ensemblistes (plusieurs modeles)
- Tester sur des taches factuelles (TriviaQA) ou le lien incertitude-exactitude est plus fort